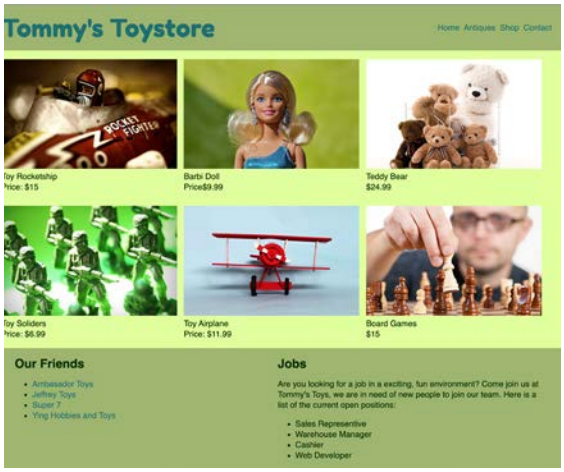
Using CSS Grids for Web Layout



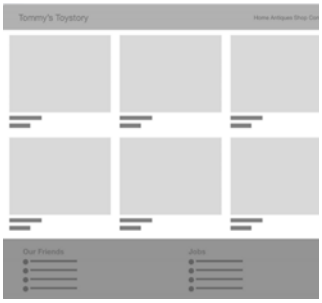
So far in this class we've discussed basic CSS visual styling such as typography (font color, font size, font family, etc) and backgrounds (background colors, background images etc), but now we are going to dive into using CSS as a layout tool.

In this lab we are going to learn about CSS grids with is a newer method for webpage layout. We will use this technique to create a sophisticated layout for our client *Tommy's Toys*.

Overview



Take a moment to open the *toystore.html* file and *style.css* file into your coding program. If you look at the HTML we have a basic webpage structure with a *<header>*, *<main>* and *<footer>*. The *<header>* tag has two *<sections>* tag nested within one another, one for the company title and one for the navigation. Inside the *<main>* each toy has it's own *<section>* tag with the toy image and the price. The footer has two major *<section> tags*, one for friends of the store and the other posting about job .



Take a moment to look at the CSS and you'll see that the fonts and backgrounds have been set. You can look at the *wireframe.pdf* to see the general layout of what we want to create. In this lab we do not need to edit the existing CSS, although we will add new CSS rules to it,

CSS Grids

What are CSS grids? CSS grids takes a HTML container and divides it into rows and columns, similar to an Excel spreadsheet. The idea of grids has existed in design and page layout for hundreds of years, and now is incorporated into CSS.



If you look are *Wireframe.pdf*, page one you'll see that the *<header>* section should be into two columns with the company name on the left and navigation bar on the right. To quickly create columns we can use CSS grids.

Let's begin by creating a CSS grid. In the CSS set the header to *display: grid* as shown below

```
header {
  display: grid;
}
```

This sets the *<header>* block to display as a *"grid"* instead of the default which is a *"block"*. When something is set to display as a grid it's direct children become the columns. In this case there are two direct children of the *<header>*, which are the two *<section>* tags, these will become the columns.

In order for the browser to know how many columns to create we must add a CSS property called *grid-template-columns*. This property allows us to specify how many columns and what their widths are.

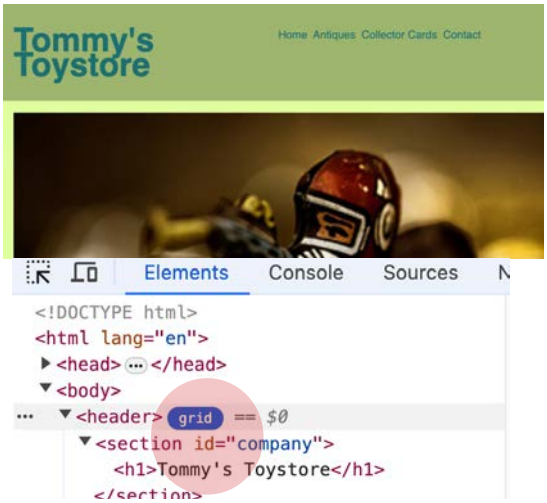
Let's create two columns that are both *400px* wide. We would write the code like this:

```
header {
  display: grid;
  grid-template-columns: 400px 400px;
}
```



This states the first column will be *400px* wide and the second column will also be *400px* wide.

Save and preview, you webpage should look roughly like the image on the right.



Most modern browsers have built-in tools to let us view the grid easily. Right click on the webpage and choose *"Inspect"*. When the Dev Tool panels opens find the *<header>* in the *Elements* tab. You will see a rounded marker that says *"grid"*. Click on the marker and the browser will preview the grid in the webpage for you.

Auto

One issue with setting the widths with finite values like 400px is that the depending on the width of the browser there may be not enough space or too much extra space. For instance, in our last example the width of the two columns is a total of 800px, but the user's browser page might be much wider, leaving alot of empty space that looks akward. A fix to this issue is to use the term auto. Auto for CSS grids means that a column does not have a specific width, but instead takes the remaining space.

Change your header CSS to look like the code below.

```
header {
  display: grid;
  grid-template-columns: 400px auto;
}
```



this means the first column will be 400px and the second column will be whatever the remaining horizontal space is.

Load the webpage into a browser. Use *Inspect* to view to Dev Tools and select the second *<section>* see that it extends to the end of the webpage on the right, as it has taken up the remaining horizontal space.



Relative Grid Values

Sometimes when laying out columns with CSS grids it is easier to set the width to relative width like *50%* as opposed to finite width like *400px* or even auto. The reason why is that it becomes easier for the webpage to readjust itself if the user makes the browser thinner or wider. Using relative values like percentages translates better begin many different devices than a finite amount like pixels. So let's change it from *400px / auto* to *50%* for each.

```
header {
  display: grid;
  grid-template-columns: 50% 50%;
}
```





Save and preview. Resize the browser thinner or wider and you will see that the columns readjust their width accordingly.

Layout the Toys

Now that we've laid out the *<header>* let's do the same for the *<main>* section which contains the toys. The *<main>* is not affected by our previous CSS grid because it is not a direct child of the *<header>*, so to affect the *<main>* let's set it to a grid as well.

```
main {
  display: grid;
}
```



If you look at *Wireframe.pdf, page one* you'll see the client was thinking about laying out all of the various toys into two columns. So let's use *grid-template-columns* and set the width to *50%* each.

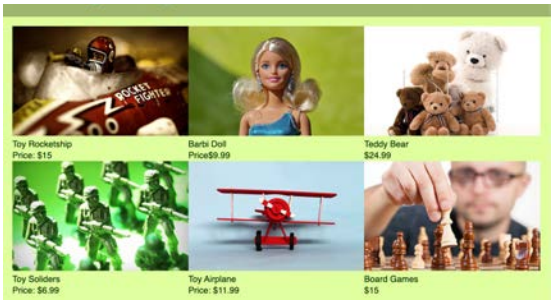
```
main {
  display: grid;
  grid-template-columns: 50% 50%;
}
```

Save and preview. You'll see you have two columns and that since the grid has six children, that you've created three rows with them. Your webpage should look like the image on the right



After the client looked at the result they decided that two columns was not working because customers could not see enough of the products, so they decided they wanted the toys to be displayed in three columns, instead of two. Believe it or not this is not too hard to fix, we'll simply go back to the CSS and change *50%* to *33.33%* and add a third column.

```
main {
  display: grid;
  grid-template-columns: 33.33% 33.33% 33.33%;
}
```



Save and preview. You see that you have three columns now instead of two.

Gaps

If you view your page you'll notice that the rows are fairly tight with each other. We can fix that by using a CSS property called *grid-row-gap*. The value is the space between our rows.

```
main {
  display: grid;
  grid-template-columns: 33.33% 33.33% 33.33%;
  grid-row-gap: 25px;
}
```



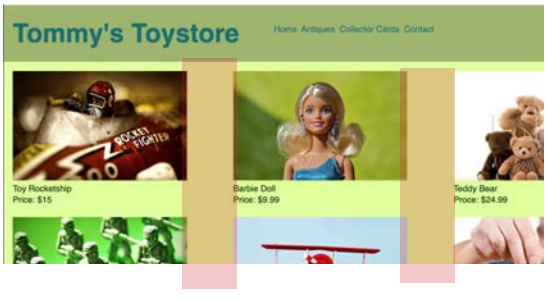
This means you'll have 25px between each row in our design.

The columns are fairly tight as well, it would be nice to have some space between each column to space out the content. We can fix this by using *grid-column-gap*. Like it sounds like it adds space between the columns as opposed to between the rows.

Add the following:

```
main {
  display: grid;
  grid-template-columns: 33.33% 33.33% 33.33%;
  grid-row-gap: 25px;
  grid-column-gap: 100px;
}
```





Save and preview. Resize the webpage thinner and you'll notice that there are now space between our columns. You will also notice that the last column is also going off of the webpage on the right. Why is this? The answer is when you add columns gaps that space is added in addition to the column space.

Fractions

At the moment we have three columns for the *<main>* that when added together take up 99.99% of the width of the webpage. It then added the two column gaps, each being 100px and then added that as well. The result is the row is now wider or above 100%, which means the last column is pushed off the side of the webpage. This is most likely not what we want, so CSS developers came up with a solution to this issue called *fractions*.

What is a fraction in CSS? A fraction is a unit of measurement, it

can be used to figure out the width of an item based on avialable space. What makes a fraction and a percentage different is that fractions will include gaps, instead of adding them afterwards like with our percentage example.

The way fractions work is specificy your columns using *fr*. Rewrite your grid-template-column like this:

```
main {
  ...
  grid-template-columns: 1fr 1fr 1fr;
  grid-column-gap: 100px;
  ...
}
```



What happens is the browser figure out that there will be three columns, each that are equal to each other, 1fr/1fr/1fr or a ratio of 1:1:1. The result is a 33% / 33% / 33% split. It will make the split **after** subtracting gaps first though, the result is that columns will never be higher than 100%, making sure your design does not go off the page.



Save and preview, and you should see that the result is the same.

Let's say we wanted to have three columns, but instead of 33/33/33 have a 50/25/25 split. What would that look like? The answer is we want a 2:1:1 ratio or 2fr / 1fr / 1fr. Set the *grid-template-column* to

```
main {
  ...
  grid-template-columns: 2fr 1fr;
  grid-column-gap: 100px
  ...
}
```



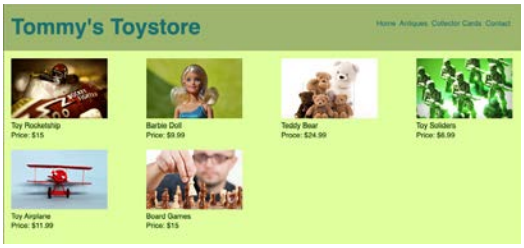
Save and you will see that the title section is twice the width of the navigation section. Let's set it back to 1:1:1 or *1fr / 1fr /1fr*.

Repeat()

When you start to use CSS grids you will notice a certain syntax being used among developers called *repeat()*. Repeat() is used to repeat the contents inside the parenthesises. For instance, change *grid-template-columns* to the following:

```
main {
  ...
  grid-template-columns: repeat(4, 1fr);
  grid-column-gap: 100px
  ...
}
```





What this is saying is there will be four columns, each being *1fr*. It would be equal to writing *1fr 1fr 1fr 1fr*. Save and you will see that the layout is divided into four equal columns. Let's change it back to our three column layout. Set the CSS code accordingly:

```
main {
  ...
  grid-template-columns: repeat(3, 1fr);
  grid-column-gap: 100px
  ...
}
```



Layout the Footer

Now you have a basic idea of how to use CSS grids and create columns set the *<footer>* to have two columns with a ratio of 1:1 or 50% each for the width of the page.



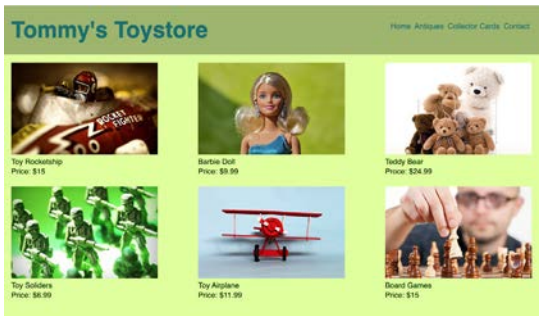
You webpage should look roughly like the image on the right.

Align to the Grid

Although we have a layout of rows and columns sometimes we'll want to position something within that column. If you look at the navigation bar you'll see it sits in the middle of the page. That is because it is sitting on left side of the right column, placing it roughly in the middle of the page. For our design we actually want the navigation bar to sit on the right side of the column. To fix this we can use previously taught concepts like *text-align*. In your CSS select the *<nav>* and use *text-align* to set it the right side of the column.



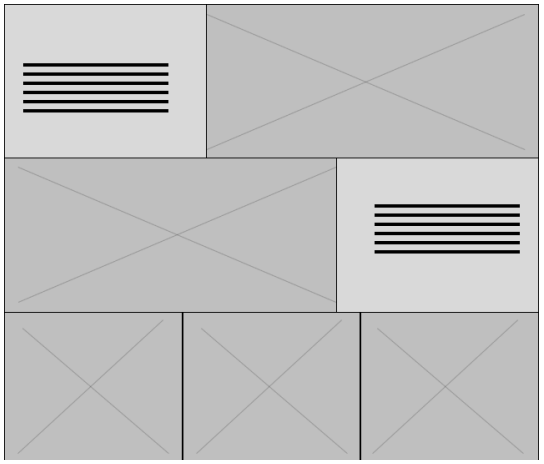

```
nav {
  text-align: right;
}
```



Save and preview

Create Mulit-Span Columns

So, at this point you should have a basic understanding of how to create grids with columns. Let's up the ante a bit with a design that has columns that are uneven, that is to say we will have one row have columns of different lengths than the row below it. We'll be applying these ideas to webpage *antiques.html*.

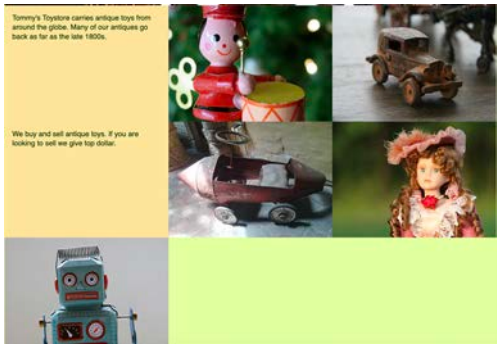


Take a moment to look over *Preview.pdf, page two* and you'll see that the in the middle of the page there are antique toys being displayed with text. In the first row the text that is roughly 1/3rd of the row and an image that takes up 2/3rds of that row, then the then second row starts with an image that is 2/3rds of the row and followed by text that is 1/3rd of the row. The final row has three

equal width columns, each with it's own image.

So it's a bit hard to see how to easily do this. Would we need to make three separate grids? We could, but that is a lot of work, instead a possible easier solution is we can make columns that span multiple columns.

Setup



In *antique.html* you'll see that we have a * * with an id of *#antiques*. Inside *#antiques* are the <section> tags we will layout with a grid.

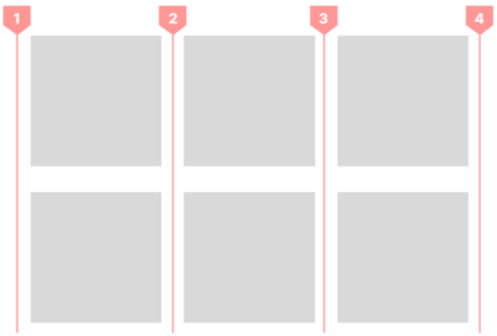
Begin by setting *#antiques* to a CSS grid and give it three columns, each being *33.33%* or *1fr*.

The result should look something like the image on the right.

Colspan

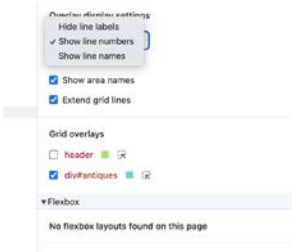
You'll notice that the columns do not look like our *Preview.pdf*. That is because each row should not have three columns each. The first row should have two columns, the second row should have two columns and the third row should have three columns.

How do fix this? The answer is a *column-span*. A column-span allows a column to span over multiple columns. For instance, if there are third columns, a grid item could span: 1 column, 2 columns or 3 columns in total. If there were four columns, the grid item could span: 1 columns, 2 columns, 3 columns or 4 columns and so on. There are a few ways to do this, both involve involve using grid tracks.



Let's look at the example at the right. Imagine I have three columns. There is a start place to the far left, that is track 1. Between the first and second column is track 2. Between the second and third columns is track 3 and at the end is track 4.

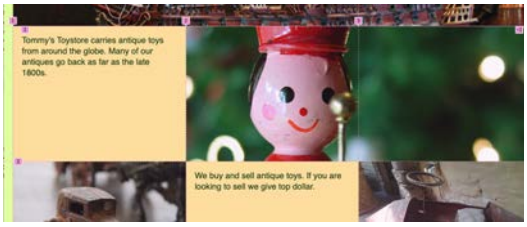
To view this in your browser go to the Developer Tools. Inside the *Elements* tab find *#antiques * and click on the rounded pill shape that says *"grid"*. If you are in Google Chrome make sure to click on the tab on the right that says *"Layout"*. Make sure the *Grid Overlay* is checked on for *div#antiques* and make sure *Overlay Display Settings* is set to *Show Line Numbers*. This will show the track number right on the webpage.



The grid item we want to span multiple columns is *#soldier* which has the background of the toy soldier. In the CSS style write the following:



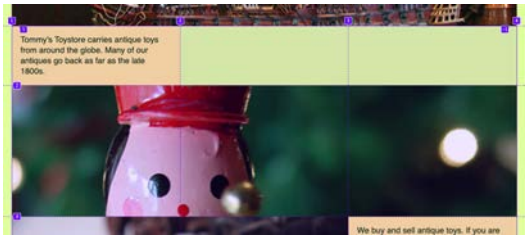
```
#soldier {
    grid-column-start: span 2;
}
```



This tells the browser to start from it's starting track, in this case 2, and go a span of 2 or two tracks. This means #splash2 will end on track 4, or the end of the grid.

What would happen if we set the span to 3?

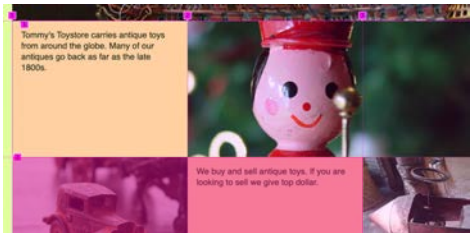
```
#soldier {
    grid-column-start: span 3;
}
```



The result is *#soldier* would be longer than the track numbers we have. If *#soldier* starts on track 2 and it is supposed to span until track 5. The problem is there is no track 5. The browser will compensate for this by bringing the *#soldier* are to the next row.

This is not really what we want, so let's set the span back to 2.

Grid-Column



Another way to do this is tell the browser what track number the grid item should start on and what track number the grid item should end on.

In the second row the toy car should span two columns. It should start on track 1 and end on track 3. We can write for this using grid-column-start and grid-column-end.

```
#toy-car {
   grid-column-start: 1;
   grid-column-end: 3;
}
```



If we wanted the *#car-toy* to go across the entire page we could write:

```
#toy-car {
   grid-column-start: 1;
   grid-column-end: 4;
}
```



Save and you will see the *#toy-car* is now expanding three columns. That actually does not work with our design so reset the *grid-column-end* to *3*.

Horizontal / Vertical Placement

In addition to creating columns it will be important for our layout to place items within the columns. This includes horizontal and vertical placement. For instance, let's say we wanted to place the Tommys Toys logo in the vertical center of the splash area at the top of the design, how would we do it? Well we could set something to the vertical center of a container by using *margin-top* or *line-height*, but what would happen if that container changes its height? Than the item inside would no longer be in the center.

The answer to this problem is using the align-content and justify-content properties that work for CSS grids. align-content allows you to align content vertically and justify-content allows you to align content horizontally.

Let's select the splash area using the *#splashSection in our CSS. Have it display as a grid.* Set the content to be placed in the vertical using *align-content: bottom.*

```
#splashSection {
  align-content: bottom;
}
```

Save and preview, you will see the logo appears at the end or bottom of the container.

Let's place the logo in the vertical center by using *align-content: center.*

```
#splashSection {
  align-content: center;
}
```

Now let's center the image in the container using *justify-content is like align-content except it places content horizontally as opposed to vertically. Let's set the logo to the horizontal center of the #splashSection by using justify-content: center;*

```
#splashSection {
  align-content: center;
  justify-content: center;
}
```

Save and preview, the logo should now appear in the horizontal as in the image on the right.

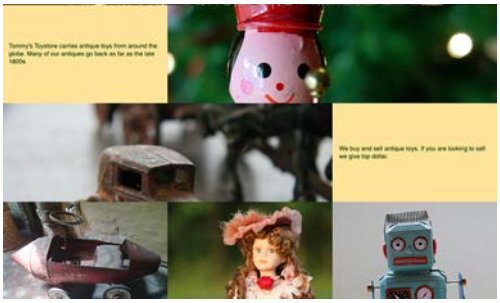
We can actually combine align-content and justify-content together into one CSS property called place-content and set that to center.



```
#splashSection {
  align-content: center;
  justify-content: center;
}
```

Use the *align-content* and *text-align* to horizontally and vertically align text for *#antique-text-1* and *#antique-text-2*.

Save and preview, your webpage should look roughly like the image on the right.



Auto-Fill

The last topic we will discuss is using CSS grids and making the designs respond to the width of the webpage. One issue that we have as developers is settings columns for multiple devices. For instance, a three column CSS grid layout might look good for a desktop computer, but as the webpage gets thinner on a smartphone the three columns of text is difficult to read. This idea of making content adjust to the device is referred to as *responsive web design* and is something we will cover in more detail in the future.

Open *collector-cards.html* into your code editor. In the HTML notice that there is a container with a class of *#collector-cards* and twelve child * * each holding a vintage baseball card.

```
#collector-cards {
  display: grid;
  grid-template-columns: 300px 300px 300px;
}
```

In the CSS document set *#collector-cards* as a CSS grids with three columns, each 300px:

#collector-cards {
  display: grid;
  grid-template-columns: 300px 300px 300px;

}



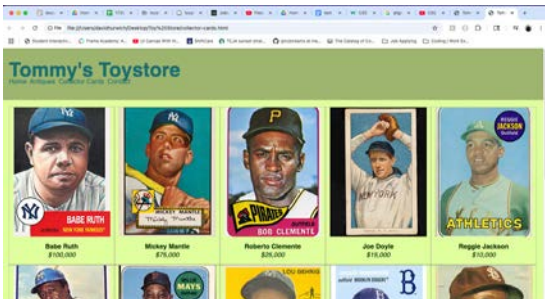
Save and preview and you will have a three column layout. If you adjust the width of the webpage thought the cards do not move and this can lead to extra space or cutting off the cards when the browser width is too tight.

To fix this we will use a CSS property called *auto-fill*. What *auto-fill* does is tell the browser to repeat the columns based on their defined width until they reach the end of the webpage. If there are more columns than space the remaining columns will wrap to the next line. This means when the webpage resizes the columns will adjust to match the available space.

```
3
4  #collector-cards {
5    display: grid;
6    grid-template-columns: repeat(auto-fill, 300px);
7  }
8
```

Rewrite the *#collector-cards* CSS to the following:

```
#collector-cards {
  display: grid;
  grid-template-columns: repeat(auto-fill, 300px);
}
```



So, what did we just write? The *repeat()* function in CSS repeats whatever is inside of the parenthesis. Auto-fill tells the browser to keep filling in columns that are 300px wide, until there is no more horizontal space and then wrap additional columns.



Save and preview. Resize your webpage and you will see that at wide screen sizes you can have more columns. As the webpage gets thinner you the number of columns lowers and the remaining cards wrap to the next line.

Resize Columns

You might notice that with this new *auto-fill* option that when there is not enough space for another column, for this lab less than 300px, that the space remains empty. It would be nice if we could have the column widths stretch out to fill in the remaining horizontal space. We can do this by tweaking our last *repeat()* function. Change the *#collector-cards* CSS to the following:

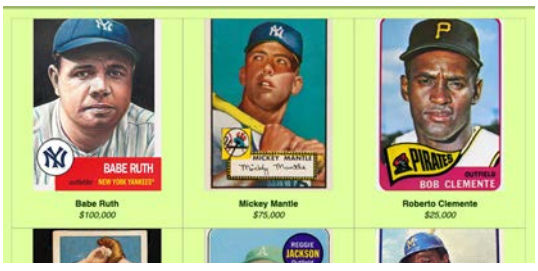
```
#collector-cards {
  display: grid;
  grid-template-columns: repeat(auto-fill, minmax(300px, 1fr);
}
```



So what exactly did we do? *Minmax()* is a CSS function that lets you say state that an HTML element can not be less than a certain value also not be higher than a certain value. In this case we are saying that the column has to be at least 300px wide, but should be larger than 1 fraction. Since it measured in fractions the space will be automatically divided between the number of columns that can fit in the that horizontal row. The result will be if there is left over space it will be added to the fraction of the columns in the row.



Save and refresh the webpage, you will see that each card column extends to take care of the remaining space. If the webpage gets thinner and there is not enough space for a card it moves to the next line and it's space is distributed to the remaining columns. Awesome!



Submission

Whew! That was a lot of cover, but hopefully this lab has helped you get a better understanding of using CSS grids as a layout tool for webpages.